

AUTOM-98 LAB: Terraform for Dynatrace

Series: AUTOM — Dynatrace Automation | **Reference:** 98 — Terraform Hands-On LAB | **Created:** April 2026 | **Last Updated:** 05/13/2026

Overview

Hands-on lab for installing the Dynatrace Terraform provider, configuring authentication, creating and managing Dynatrace resources as code, importing existing configuration, and integrating into a CI/CD pipeline with GitHub Actions.

Table of Contents

1. [Install Terraform](#)
 2. [Configure the Dynatrace Provider](#)
 3. [Create Your First Resource — Alerting Profile](#)
 4. [Create Settings 2.0 Resources](#)
 5. [Create IAM Resources \(OAuth Required\)](#)
 6. [Import Existing Resources](#)
 7. [Variables and Multi-Environment](#)
 8. [State Management](#)
 9. [GitHub Actions CI/CD Pipeline](#)
 10. [Drift Detection](#)
 11. [Summary and Checklist](#)
-

Prerequisites

Requirement	Details
Completed	AUTOM-04: Terraform Provider (lecture notebook)
Dynatrace Environment	SaaS tenant (Gen3)
Terraform CLI	Version 1.0+ installed
Authentication	API Token and/or OAuth Client credentials
Permissions	Settings write, IAM admin (for Section 5)
Git	Git CLI and a GitHub account (for Section 9)

1. Install Terraform

Terraform is a single binary. Download it from terraform.io or use a package manager.

macOS (Homebrew):

```
brew tap hashicorp/tap
brew install hashicorp/tap/terraform
```

Linux (APT — Debian/Ubuntu):

```
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee
/etc/apt/sources.list.d/hashicorp.list
sudo apt-get update &&& sudo apt-get install terraform
```

Windows (Chocolatey):

```
choco install terraform
```

Verify the installation:

```
terraform version
# Expected output: Terraform v1.x.x
```

Note: The Dynatrace Terraform provider requires Terraform 1.0 or later. Any 1.x release will work.

2. Configure the Dynatrace Provider

Create a project directory and a `main.tf` file:

```
mkdir dynatrace-terraform && cd dynatrace-terraform
touch main.tf
```

Add the provider block to `main.tf`:

```
terraform {
  required_providers {
    dynatrace = {
      source  = "dynatrace-oss/dynatrace"
      version = "~> 1.96"
    }
  }
}
```

Token-type decision (read first): The provider has a **mixed-auth model** — some resources need a Platform Token (`dt0s16`), others need a classic API Token (`dt0c01`), and IAM resources need OAuth. For the full per-resource decision matrix, see **AUTOM-04 § 3 "Service User Credentials for Terraform — Platform Token vs Classic API Token"**. The methods below cover the three credential types you will configure; the lecture explains *which one each resource needs*.

Authentication Method 1: Platform Token (default for most new resources)

Best for Settings 2.0, Gen3 resources (workflows, documents, segments, buckets, OpenPipeline). Mint in the Dynatrace UI under **Account Management > Identity & access management > Platform tokens** with the scopes the resource requires.

```
provider "dynatrace" {  
  dt_env_url      = "https://&lt;env-id&gt;.apps.dynatrace.com"  
  platform_token  = var.dt_platform_token  
}
```

Authentication Method 2: Classic API Token (legacy + a few specific resources)

Required for Synthetic monitors, SLO v1, and the `dynatrace_api_token` resource itself. Often configured **alongside** a Platform Token in the same provider block.

```
provider "dynatrace" {  
  dt_env_url      = "https://&lt;env-id&gt;.live.dynatrace.com"  
  dt_api_token    = var.dt_api_token  
  platform_token  = var.dt_platform_token  
}
```

Important (v1.88.0+): The OAuth functionality was removed from ~16 provider resources in v1.88.0. Synthetic monitors, SLO v1, and `dynatrace_api_token` no longer accept OAuth — they need a classic API Token. See AUTOM-04 § 3 for the full list.

Authentication Method 3: OAuth Client Credentials (IAM resources only)

Required for `dynatrace_iam_*` resources (groups, policies, bindings).

```
provider "dynatrace" {  
  dt_env_url      = "https://&lt;env-id&gt;.apps.dynatrace.com"  
  dt_client_id    = var.dt_client_id  
  dt_client_secret = var.dt_client_secret  
  dt_account_id   = var.dt_account_id  
}
```

Authentication Method 4: Environment Variables

Keep credentials out of `.tf` files entirely:

```
export DYNATRACE_ENV_URL="https://&lt;env-id&gt;.apps.dynatrace.com"  
export DT_PLATFORM_TOKEN="dt0s16.xxx..."  
export DYNATRACE_API_TOKEN="dt0c01.xxx..."
```

With environment variables set, the provider block needs no arguments:

```
provider "dynatrace" {}
```

Production callout — run Terraform as a Service User. Outside this LAB, mint the Platform Token (and any classic API Token) on a **Service User**, not on a human's account. Three things must align: the Service User's IAM permissions, the creator's `iam:service-users:use` permission, and the token scopes at creation time. The `dynatrace_api_token` resource also stores its minted token plain-text in state — see **AUTOM-04 § 3 "Operational Safety — State File Leakage"** and the **"Bridging the Trust Boundary"** subsection for the recommended *mint-out-of-band, deliver-in-band* pattern.

Initialize the Provider

Run `terraform init` to download the provider plugin:

```
terraform init  
# Initializing provider plugins...  
# - Installing dynatrace-oss/dynatrace v1.96.x...  
# Terraform has been successfully initialized!
```

3. Create Your First Resource — Alerting Profile

Add an alerting profile resource to `main.tf` :

```
resource "dynatrace_alerting" "production" {
  name = "Production Alerts"
  rules {
    rule {
      severity_level = "AVAILABILITY"
      delay_in_minutes = 0
    }
    rule {
      severity_level = "ERROR"
      delay_in_minutes = 5
    }
  }
}
```

Preview the Change

```
terraform plan
```

Expected output:

```
Terraform will perform the following actions:

# dynatrace_alerting.production will be created
+ resource "dynatrace_alerting" "production" {
+   id      = (known after apply)
+   name    = "Production Alerts"
+   rules {
+     rule {
+       severity_level = "AVAILABILITY"
+       delay_in_minutes = 0
+     }
+     rule {
+       severity_level = "ERROR"
+       delay_in_minutes = 5
+     }
+   }
+ }

Plan: 1 to add, 0 to change, 0 to destroy.
```

Apply the Change

```
terraform apply
# Type "yes" when prompted to confirm
```

Verify the Resource

```
terraform show
```

Confirm the alerting profile exists in your Dynatrace tenant under **Settings > Alerting > Alerting profiles**.

4. Create Settings 2.0 Resources

The `dynatrace_generic_setting` resource can manage any Settings 2.0 schema. This is useful when the provider does not have a dedicated resource type for a particular setting.

Example: Kubernetes Metadata Enrichment Rule

```
resource "dynatrace_generic_setting" "k8s_enrichment" {
  schema_id = "builtin:kubernetes.generic.metadata.enrichment"
  scope     = "environment"
  value = jsonencode({
    enabled = true
    rules = [{
      sourceAttribute = "namespace-label"
      sourceKey       = "team"
      targetAttribute = "dt.security_context"
    }]
  })
}
```

Key Attributes

Attribute	Description
schema_id	The Settings 2.0 schema identifier (e.g., <code>builtin:kubernetes.generic.metadata.enrichment</code>)
scope	Where the setting applies: <code>environment</code> , <code>HOST-xxx</code> , <code>KUBERNETES_CLUSTER-xxx</code> , etc.
value	JSON-encoded configuration object matching the schema definition

Finding Schema IDs

Navigate to **Settings** in the Dynatrace UI. The schema ID appears in the URL when you open any setting:

```
https://&lt;env-  
id>.apps.dynatrace.com/ui/settings/builtin:kubernetes.generic.metadata.enr  
ichment
```

```

-----
This is the schema_id

```

Alternatively, use the Settings API:

```
curl -H "Authorization: Api-Token $DT_API_TOKEN" \
  "https://&lt;env-id&gt;.live.dynatrace.com/api/v2/settings/schemas" | jq
'.items[].schemaId'
```

5. Create IAM Resources (OAuth Required)

IAM resources (groups, policies, bindings) require OAuth client credentials. API tokens cannot manage IAM.

Create an IAM Group

```
resource "dynatrace_iam_group" "sre_team" {  
  name = "SRE Team"  
}
```

Create an IAM Policy

```
resource "dynatrace_iam_policy" "sre_policy" {  
  name           = "SRE Full Access"  
  statement_query = "ALLOW environment:roles:viewer;"  
  tags           = ["sre"]  
}
```

Bind the Policy to the Group

```
resource "dynatrace_iam_policy_bindings_v2" "sre_binding" {  
  group_id   = dynatrace_iam_group.sre_team.id  
  account_id = var.dt_account_id  
  policies   = [dynatrace_iam_policy.sre_policy.id]  
}
```

Apply

```
terraform plan  
terraform apply
```

Note: The `account_id` variable is required for IAM policy bindings. This is your Dynatrace account UUID, found under **Account Management > Account settings**.

6. Import Existing Resources

If you already have Dynatrace resources configured manually, you can bring them under Terraform management without recreating them.

Bootstrapping from an existing tenant in bulk? Use the provider's built-in `-export utility` rather than running `terraform import` per-resource. From a downloaded provider binary: `./terraform-provider-dynatrace -export -env <url> -token <token>` generates `.tf` files for the entire tenant. The per-resource flow below is right when you only need to onboard a few specific resources.

Step 1: Add a Resource Block

Create an empty resource block in your `.tf` file:

```
resource "dynatrace_alerting" "existing_profile" {
  # Configuration will be filled after import
}
```

Step 2: Find the Resource ID

Locate the resource ID from the Dynatrace UI or API. For alerting profiles, it appears in the URL:

[illegible]

Step 3: Import

```
terraform import dynatrace_alerting.existing_profile abc12345-def6-7890-abcd-ef1234567890
```

Step 4: Generate the HCL

After importing, extract the configuration from state:

```
terraform show -no-color > imported.tf
```

Review and clean up the generated HCL. Remove read-only attributes (like `id`) and format the code.

Common Import Targets

Resource Type	ID Source
<code>dynatrace_alerting</code>	Settings UI URL or API
<code>dynatrace_dashboard</code>	Dashboard URL (<code>id</code> parameter)
<code>dynatrace_slo</code>	SLO list page or API
<code>dynatrace_generic_setting</code>	Settings API (<code>objectId</code> field)

Tip: After import, run `terraform plan` to verify the imported state matches the live configuration. Any differences indicate drift that you should reconcile in the `.tf` file.

7. Variables and Multi-Environment

Define Variables

Create `variables.tf` :

```
variable "dt_env_url" {
  type      = string
  description = "Dynatrace environment URL"
}

variable "dt_api_token" {
  type      = string
  sensitive  = true
  description = "Dynatrace API token"
}

variable "environment" {
  type      = string
  default    = "dev"
  description = "Target environment name (dev, staging, production)"
}
```

Supply Values with tfvars

Create `terraform.tfvars` (add to `.gitignore` — never commit credentials):

```
dt_env_url    = "https://<env-id>.live.dynatrace.com"
dt_api_token  = "dt0c01.xxx..."
environment   = "production"
```

Approach 1: Workspace-Based Multi-Environment

Terraform workspaces maintain separate state files per environment using the same configuration:

```
# Create workspaces
terraform workspace new dev
terraform workspace new staging
terraform workspace new production

# Switch between environments
terraform workspace select production

# Use workspace name in configuration
# terraform.workspace returns the current workspace name
```

Reference the workspace in your resources:

```
resource "dynatrace_alerting" "alerts" {
  name = "${terraform.workspace}-alerts"
  # ...
}
```

Approach 2: Directory-Based Multi-Environment

For larger teams, use separate directories with shared modules:

```
dynatrace-terraform/
modules/
  alerting/
    main.tf
    variables.tf
environments/
  dev/
    main.tf          # calls module with dev values
    terraform.tfvars
  staging/
    main.tf
    terraform.tfvars
  production/
    main.tf
    terraform.tfvars
```

Each environment directory runs independently with its own state and variables.

8. State Management

Terraform state tracks the mapping between your `.tf` files and real Dynatrace resources. By default, state is stored locally in `terraform.tfstate`.

State-file leakage warning. If your Terraform manages the `dynatrace_api_token` resource, the minted token value is written **plain-text** to state regardless of `sensitive = true`. The same risk applies to any provider that produces a credential at apply time. Mitigate with backend encryption + restricted access — see **AUTOM-04 § 3 "Operational Safety — State File Leakage When Minting**

Tokens" for the full pattern, and **AUTOM-09 § 3** (state backends) and **§ 8** (secrets handling) for the bootstrap recipe.

Local State (Default)

Works for single-operator setups. The state file is created automatically after `terraform apply`.

Warning: Never commit `terraform.tfstate` to version control. It may contain sensitive values. Add it to `.gitignore`.

Remote State — S3 Backend

For team use, store state in a shared remote backend. The S3 backend has supported **native locking via S3 conditional writes** since Terraform 1.10 — no DynamoDB table required:

```
terraform {
  backend "s3" {
    bucket      = "my-terraform-state"
    key         = "dynatrace/terraform.tfstate"
    region      = "us-east-1"
    encrypt     = true
    use_lockfile = true    # native S3 locking (Terraform 1.10+); DynamoDB no
                           # longer required
  }
}
```

Attribute	Purpose
<code>bucket</code>	S3 bucket for state storage
<code>key</code>	Path within the bucket
<code>use_lockfile</code>	Native S3 lockfile-based concurrency control (Terraform 1.10+); replaces DynamoDB-based locking
<code>encrypt</code>	Encrypt state at rest (use SSE-KMS with a customer-managed key for production)

Migrating from DynamoDB? Set `use_lockfile = true` alongside the existing `dynamodb_table` attribute, run `terraform init -reconfigure`, verify lock acquisition succeeds, then remove `dynamodb_table` on a follow-up apply. See AUTOM-09 § 3 for the full migration recipe.

Remote State — Azure Blob

```
terraform {
  backend "azurerm" {
    resource_group_name = "rg-terraform"
    storage_account_name = "tfstateaccount"
    container_name       = "tfstate"
    key                  = "dynatrace.terraform.tfstate"
  }
}
```

Remote State — Terraform Cloud / HCP Terraform

```
terraform {
  cloud {
    organization = "my-org"
    workspaces {
      name = "dynatrace-production"
    }
  }
}
```

Inspecting State

```
# List all resources in state
terraform state list

# Show details of a specific resource
terraform state show dynatrace_alerting.production

# Remove a resource from state (without destroying it)
terraform state rm dynatrace_alerting.production
```

9. GitHub Actions CI/CD Pipeline

This section is a **minimal starter** to confirm the basics work end-to-end with static GitHub Secrets. For production CI/CD — OIDC-federated secret retrieval, multi-environment promotion, plan-comments on PRs, manual-approval gates, and patterns for GitLab / Bitbucket / Bamboo / Jenkins / Azure DevOps — go to:

Where	What it covers
AUTOM-96 LAB: GitHub Actions CI/CD for Dynatrace Terraform	Full hands-on: OIDC → Vault → Platform Token, plan-comment via <code>actions/github-script</code> , environment-gated apply, validation walkthrough. The recommended-default end state.
AUTOM-07: CI/CD Integration	Conceptual coverage of GitHub Actions, GitLab CI, Bitbucket Pipelines, Atlassian Bamboo, Jenkins, Azure DevOps.
AUTOM-04 § 3 "Bridging the Trust Boundary"	Why static GitHub Secrets is no longer the recommended default, and the five mechanisms for getting tokens to the runner.

The starter below uses a static `secrets.DT_API_TOKEN` and is intentionally kept simple — do **not** ship this pattern to production.

Starter Workflow (static secrets — not for production)

Create `.github/workflows/terraform-deploy.yml` :


```
name: Deploy Dynatrace Terraform (starter)

on:
  pull_request:
    paths: ["terraform/**"]
  push:
    branches: [main]
    paths: ["terraform/**"]

jobs:
  plan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3

      - name: Terraform Init
        run: terraform init
        working-directory: terraform/

      - name: Terraform Plan
        run: terraform plan -no-color
        working-directory: terraform/
        env:
          DYNATRACE_ENV_URL: ${ secrets.DT_ENV_URL }
          DYNATRACE_API_TOKEN: ${ secrets.DT_API_TOKEN }

  apply:
    if: github.ref == 'refs/heads/main' && github.event_name == 'push'
    needs: plan
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3

      - name: Terraform Init
        run: terraform init
        working-directory: terraform/

      - name: Terraform Apply
        run: terraform apply -auto-approve
        working-directory: terraform/
        env:
          DYNATRACE_ENV_URL: ${ secrets.DT_ENV_URL }
          DYNATRACE_API_TOKEN: ${ secrets.DT_API_TOKEN }
```

GitHub Secrets (starter only)

Store credentials in GitHub repository secrets (**Settings > Secrets and variables > Actions**):

Secret Name	Value
DT_ENV_URL	https://<env-id>.live.dynatrace.com
DT_API_TOKEN	Your Dynatrace API token

Production migration path: Once the starter runs cleanly, walk through **AUTOM-96 LAB** to replace these static secrets with OIDC-federated runtime credential fetch from Vault (or AWS Secrets Manager / Azure Key Vault / GCP Secret Manager). The end state has zero long-lived Dynatrace credentials in GitHub Secrets.

10. Drift Detection

Drift occurs when someone changes a Dynatrace resource manually (through the UI or API) after it was deployed via Terraform.

Detect Drift

```
# Run plan with detailed exit code
terraform plan -detailed-exitcode

# Exit codes:
# 0 = No changes (no drift)
# 1 = Error
# 2 = Changes detected (drift found)
```

Scheduled Drift Detection

Add a scheduled GitHub Actions workflow to check for drift daily:

```

name: Drift Detection

on:
  schedule:
    - cron: "0 8 * * 1-5"    # Weekdays at 8 AM UTC

jobs:
  detect-drift:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
      - run: terraform init
        working-directory: terraform/
      - name: Check for drift
        id: drift
        run: terraform plan -detailed-exitcode
        working-directory: terraform/
        continue-on-error: true
        env:
          DYNATRACE_ENV_URL: ${ secrets.DT_ENV_URL }
          DYNATRACE_API_TOKEN: ${ secrets.DT_API_TOKEN }
      - name: Alert on drift
        if: steps.drift.outcome == 'failure'
        run: echo "Drift detected! Review terraform plan output."

```

Handling Drift

Scenario	Action
Manual change was intentional	Update <code>.tf</code> to match, or <code>terraform import</code> the new state
Manual change was accidental	Run <code>terraform apply</code> to restore the desired state
State is stale	Run <code>terraform refresh</code> to update state from the live tenant

11. Summary and Checklist

Deployment Checklist

- [] Terraform 1.0+ installed and verified (1.10+ recommended for native S3 backend locking)
- [] Provider configured with the **right token type per resource** (Platform / classic API / OAuth — see AUTOM-04 § 3)
- [] `terraform init` downloads the Dynatrace provider (v1.96.x or later)
- [] Resources defined in `.tf` files, not configured manually
- [] `terraform.tfvars` and `terraform.tfstate` in `.gitignore`
- [] Remote state backend configured for team use, with encryption at rest
- [] **Production:** token holder is a Service User, not a human account (see AUTOM-04 § 3)
- [] **Production:** no `dynatrace_api_token` minted in this state file (or backend access is locked down — state stores the token plain-text)
- [] CI/CD pipeline runs `plan` on PRs, `apply` on merge — production uses OIDC-federated secret fetch (see AUTOM-96)
- [] Drift detection scheduled

Key Takeaways

Concept	Key Point
Provider Setup	Pin version with <code>~> 1.96</code> ; v1.88.0+ uses a mixed-auth model
Authentication	Platform Token (default) + classic API Token (Synthetics, SLO v1, <code>dynatrace_api_token</code>) + OAuth (IAM only) — see AUTOM-04 § 3
Service User	Production token holder is a Service User; three things must align (perms, creator scope, token scope)
Plan Before Apply	Always review <code>terraform plan</code> output before applying
State Management	Use remote backends with encryption; <code>dynatrace_api_token</code> stores its value plain-text in state regardless of <code>sensitive</code>

Concept	Key Point
Import	Use <code>-export</code> utility for bulk bootstrap; <code>terraform import</code> for individual resources
CI/CD	Static GitHub Secrets is a starter; production uses OIDC → Vault (or cloud-native secret manager) — see AUTOM-96
Drift Detection	Schedule regular checks with <code>plan -detailed-exitcode</code>

Monaco vs Terraform — When to Use Which

Scenario	Recommended Tool
Quick config export/import	Either (Monaco UI flow, or Terraform <code>-export</code> utility)
Multi-environment promotion	Either
IAM policy management	Terraform (OAuth required)
Multi-cloud infrastructure + Dynatrace	Terraform
Drift detection and state tracking	Terraform
Team with existing Terraform expertise	Terraform
Team new to IaC	Monaco (lower learning curve)

References

Resource	URL
Terraform Registry	registry.terraform.io/providers/dynatrace-oss/dynatrace
Provider Documentation	registry.terraform.io/providers/dynatrace-oss/dynatrace/latest/docs
Provider GitHub	github.com/dynatrace-oss/terraform-provider-dynatrace
Terraform Install	developer.hashicorp.com/terraform/install

Resource	URL
Dynatrace IAM Docs	docs.dynatrace.com/docs/manage/identity-access-management

Continue to AUTOM-09: Terraform GitOps Setup Recipe for state backends, lifecycle protections, and multi-environment promotion; then AUTOM-07: CI/CD Integration for production pipeline patterns (and the AUTOM-96 LAB for the GitHub Actions hands-on); or jump to AUTOM-05: Dynatrace Workflows for event-driven automation.

Community Resources

Repository	Description
terraform-provider-dynatrace	Official provider (v1.96.x) with built-in <code>-export</code> utility
dynatrace-configuration-as-code-samples	10 starter templates in <code>basic-templates-terraform</code> , plus modules, IAM, and DQL examples
terraform_modules	Reusable module pattern for synthetic monitors
terraform_dql_example	DQL as a Terraform data source for dynamic config generation
terraform_team_onboarding	IAM policies, groups, and Azure Entra ID for team provisioning
iam_tf_sample	IAM policies, Grail buckets, OpenPipelines, and segments

Tip: Use the provider export feature (`./terraform-provider-dynatrace -export -env <url> -token <token>`) to generate `.tf` files from an existing tenant — the fastest path to Terraform-managed configuration.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official

Dynatrace documentation.